

Endurecimento Pós-Quântico do SSH em 2026: A Transição para mlkem768x25519-sha256 no OpenSSH

Day 03 | Category: Endurecimento de Protocolo de Rede

Date: 2026-07-10 |

DAILY MONITORING

Na transição para a resistência pós-quântica, proteger os canais administrativos é tão crucial quanto proteger o tráfego web normal. O **Secure Shell (SSH)**, que é a espinha dorsal da administração moderna de servidores e nuvem, passou por uma rápida integração com a Criptografia Pós-Quântica (PQC). Enquanto as versões anteriores do OpenSSH dependiam de opções híbridas experimentais (como o `snttrup761x25519-sha512`), implementações modernas como o **OpenSSH 9.9 (e mais recentes)** e sistemas como o **Ubuntu 26.04 LTS** estabeleceram o algoritmo `mlkem768x25519-sha256` como o mecanismo padrão e preferencial de troca de chaves (KEX).

Princípio de Segurança Subjacente: O algoritmo de troca de chaves `mlkem768x25519-sha256` é um mecanismo híbrido. Ele combina o **ML-KEM-768** (o mecanismo de encapsulamento de chave baseado em reticulados do padrão oficial NIST FIPS 203) com a curva elíptica clássica Diffie-Hellman **X25519**, vinculando-os através de uma KDF baseada em SHA-256. Este design "híbrido" garante que se o ML-KEM vier a ser quebrado por futuras descobertas matemáticas, a conexão permanecerá tão segura quanto uma sessão clássica com X25519.

A Evolução da Troca de Chaves Pós-Quântica no OpenSSH

Para entender o estado atual, devemos analisar como o OpenSSH gerenciou essa transição pós-quântica:

- **OpenSSH 8.5 até 9.8:** Introduziu e definiu como padrão o `snttrup761x25519` (combinando Streamlined NTRU Prime 761 e X25519) porque o NTRU Prime era considerado uma escolha conservadora e imune a questões de patentes. No entanto, não foi o algoritmo final escolhido pelo NIST.
- **OpenSSH 9.9:** Após a publicação oficial do padrão FIPS 203 pelo NIST no final de 2024, o OpenSSH introduziu o `mlkem768x25519-sha256`, tornando-o o algoritmo de troca de chaves de maior prioridade, e removeu algoritmos obsoletos como o DSA.
- **Ubuntu 26.04 LTS (2026):** De forma nativa, o cliente e o servidor SSH do Ubuntu 26.04 negociam o `mlkem768x25519-sha256` por padrão, garantindo sessões administrativas seguras contra ameaças quânticas desde o primeiro dia de instalação.

Configuração e Endurecimento (Hardening) do OpenSSH para PQC

Por padrão, o OpenSSH negocia o algoritmo mais forte suportado por ambas as pontas. Contudo, para atender a requisitos rigorosos de segurança pós-quântica e eliminar riscos de queda para algoritmos clássicos, os administradores podem forçar uma política estrita que aceita apenas trocas de chaves pós-quânticas.

Auditando o Algoritmo Negociado

Para inspecionar qual algoritmo de troca de chaves foi negociado em uma sessão ativa, conecte-se usando o modo duplo-detalhado (-vv) e filtre as linhas de negociação (kex):

```
ssh -vv usuario@servidor exit 2>&1 | grep "kex:"
```

Se o cliente e o servidor estiverem atualizados, você verá a confirmação na saída:

```
debug1: kex: algorithm: mlkem768x25519-sha256
```

Forçando KEX Apenas Pós-Quântico

Para restringir seu servidor SSH a aceitar somente trocas de chaves pós-quânticas ou híbridas, edite o arquivo de configuração do daemon (por exemplo, em /etc/ssh/sshd_config.d/99-pqc.conf):

Trecho de Configuração:

```
# Forçar apenas trocas de chave híbridas pós-quânticas
KexAlgorithms mlkem768x25519-sha256,sntrup761x25519-sha512

# Habilitar logs detalhados para auditar capacidades dos clientes
LogLevel VERBOSE
```

Antes de reiniciar o serviço, sempre valide a sintaxe dos arquivos para evitar perder o acesso remoto ao servidor: `sudo sshd -t`. Uma vez validado, reinicie o daemon com `sudo systemctl restart ssh`.

Comparação de Algoritmos e Desempenho no SSH

A tabela a seguir apresenta os principais algoritmos de troca de chaves disponíveis nos daemons SSH modernos e seus respectivos parâmetros operacionais:

Nome do Algoritmo	Tipo	Sobrecarga da Troca de Chaves (Bytes)	Fundamentação Criptográfica	Resistente a Quântico?	Status de Produção (2026)
curve25519-sha256	ECDH Clássico	64	Curve25519 (Logaritmo Discreto)	Não	Apenas Fallback
sntrup761x25519-sha512	KEM Híbrido	~1.200	NTRU Prime + X25519	Sim (Híbrido)	Suportado (Legado PQ)
mlkem768x25519-sha256	KEM Híbrido	~2.300	ML-KEM-768 (M-LWE) + X25519	Sim (Híbrido)	Padrão Atual

Quiz Diário: Teste seu Conhecimento

1. Por que o `mlkem768x25519-sha256` é preferido em relação ao `snttrup761x25519-sha512` no OpenSSH 9.9 e em sistemas operacionais modernos como o Ubuntu 26.04?

A) Porque o NTRU Prime foi totalmente quebrado por computadores lineares clássicos.
B) Porque o ML-KEM-768 é o padrão oficial do NIST (FIPS 203), enquanto o NTRU Prime não foi selecionado no portfólio final do NIST.
C) Porque o ML-KEM-768 possui chaves públicas significativamente menores do que a `curve25519`.

Resposta Correta: B. Embora o NTRU Prime seja um esquema seguro e respeitável (e tenha sido muito útil como solução temporária), o ML-KEM é o algoritmo pós-quântico oficial definido na publicação FIPS 203 do NIST, tornando-se a linha de base de conformidade aceita globalmente.

2. Qual é o principal risco de não configurar uma troca de chaves pós-quântica em um servidor SSH atualmente?

A) Invasores podem invadir instantaneamente a sessão SSH usando computadores quânticos comerciais já em 2026.
B) Clientes SSH antigos não conseguirão mais se conectar ao servidor.
C) Sessões ficam vulneráveis a ataques do tipo "Harvest Now, Decrypt Later" (Coletar Agora, Decifrar Depois), em que as sessões administrativas cifradas são gravadas hoje e decifradas no futuro quando um computador quântico viável surgir.

Resposta Correta: C. As chaves de sessão simétricas acordadas usando Diffie-Hellman ou ECDH clássicos são vulneráveis à decifragem retrospectiva via algoritmo de Shor quando um computador quântico potente o suficiente estiver disponível.

3. Qual comando é usado para verificar com segurança a configuração do servidor SSH após endurecê-lo com algoritmos apenas pós-quânticos?

A) `sudo systemctl restart ssh`
B) `sudo sshd -t`
C) `ssh-keygen -A`

Resposta Correta: B. O comando `sudo sshd -t` testa os arquivos de configuração em busca de erros de sintaxe sem reiniciar o daemon SSH. Executar este comando evita que você perca o acesso a um servidor remoto devido a um erro de digitação.